

Major: ELECTRONICS ENGINEERING

Project: FSK ROBOTIC ARM CONTROL

Oscar Yair Olvera López

Santiago de Querétaro, 06 de junio de 2026

índice

1. Objetivo Específico	¡Error! Marcador no definido.
2. Introducción.....	¡Error! Marcador no definido.
3. Marco Teórico	4
4. Equipo y materiales	4
5. Seguridad.....	8
6. Desarrollo de la práctica.....	8
6.1. Etapa de Visión (LabVIEW)	8
6.2. Etapa de Modulación (Transmisor TX).....	8
6.3. Etapa de Demodulación y Control (Receptor RX)	17
7. Resultados	21
8. Conclusiones y recomendaciones	23
9. Referencias bibliográficas	24

1. Specific Objective

Design and implement an integrated electromechanical system capable of sorting objects using machine vision (QR codes), transmitting the control decision over an analog telecommunications link via FSK (Frequency Shift Keying) modulation using the XR2206 and XR2211 integrated circuits, and decoding the data frame via software to kinematically actuate a 4-Degree-of-Freedom (DOF) robotic arm using the I2C protocol.

2. Introduction

In industrial automation, it is common for the heavy processing stage (such as machine vision) to be physically separated from the mechanical actuation stage. Transmitting pure digital signals over long distances in industrial environments is highly susceptible to degradation from electromagnetic noise.

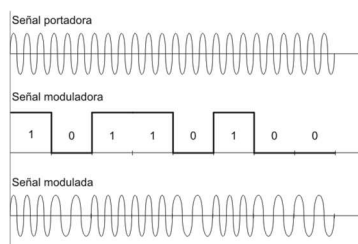
To solve this problem, the present project implements a hybrid instrumentation system. A computer processes the images and sends ASCII characters. These digital data are converted into audio frequency variations (FSK) by a Voltage-Controlled Oscillator (VCO), traveling through a robust analog bus. At the receiving end, a Phase-Locked Loop (PLL) recovers the digital data, and a microcontroller interprets the frame to execute parameterized trajectories on a robotic manipulator.



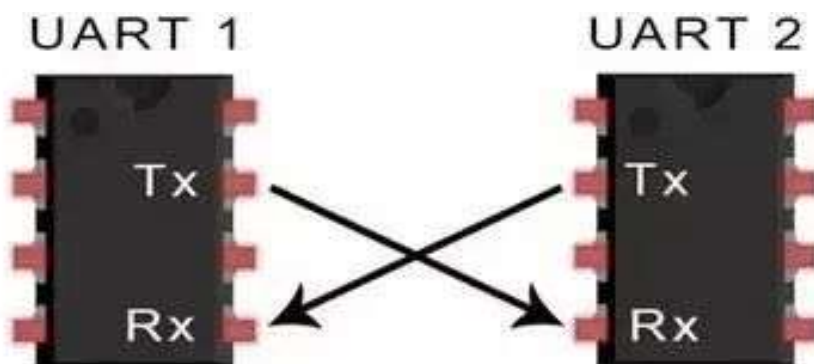
3. Theoretical Framework

3.1. Modulación FSK y Emulación UART

FSK (Frequency Shift Keying) modulation is a digital transmission technique where binary information is represented by frequency changes in an analog carrier signal. In this project, a logical "1" (mark) corresponds to a frequency of 2000 Hz, and a logical "0" (space) to a frequency of 1000 Hz. The main advantage of FSK over Amplitude Shift Keying (ASK) is its high immunity to noise and line attenuation variations.



To send structured data, the UART (Universal Asynchronous Receiver-Transmitter) protocol is emulated via software ("Bit-banging"). This protocol requires the line to be held at a HIGH level during its idle state. To transmit an ASCII letter, a falling edge or Start Bit (LOW) is sent, followed by 8 data bits (from the most significant bit to the least significant bit), and concluded with a Stop Bit (HIGH). The agreed transmission rate is approximately 16 baud, which equals exactly 62.5 milliseconds per bit.



3.2. XR2206 Circuit (VCO Modulator)

The XR2206 is a monolithic function generator. Its core is a Voltage-Controlled Oscillator (VCO) whose oscillation frequency is inversely proportional to the value of a timing capacitor (C) and a resistor (R).

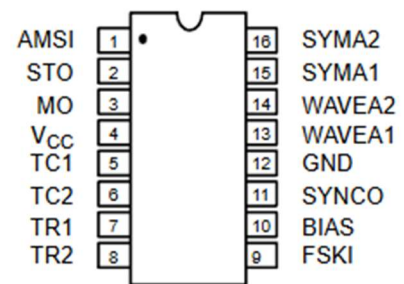
The circuit allows quick toggling between two stable frequencies depending on the logical voltage applied to its FSK input (Pin 9):

- If Pin 9 detects HIGH, the frequency is governed by resistor R1.
- If Pin 9 detects LOW, the frequency shifts to the R2 resistor circuit.

The XR2206 is a monolithic function generator. Its core is a Voltage-Controlled Oscillator (VCO) whose oscillation frequency is inversely proportional to the value of a timing capacitor (C) and a resistor (R).

The circuit allows quick toggling between two stable frequencies depending on the logical voltage applied to its FSK input (Pin 9):

- If Pin 9 detects HIGH, the frequency is governed by resistor R1.
- If Pin 9 detects LOW, the frequency shifts to the R2 resistor circuit.



16 Lead PDIP, CDIP (0.300")

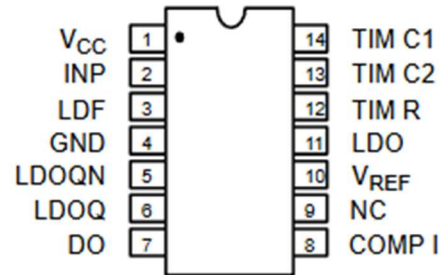
3.3. XR2211 Circuit (PLL Demodulator)

The XR2211 is an FSK decoder based on a Phase-Locked Loop (PLL). The PLL consists of three fundamental stages:

1. **Phase Detector:** Compares the phase of the incoming analog signal with the phase of its local oscillator.
2. **Loop Filter:** Filters the error voltage resulting from the phase detector, removing high-frequency noise and determining the system's dynamic response time.
3. **Internal VCO:** Adjusts its own frequency to match (or "lock" onto) the input signal. Once locked, an internal comparator toggles the logic state of its output (Pin 7) depending on whether the input frequency is higher or lower than the oscillator's free-running frequency.

The XR2211 is an FSK decoder based on a Phase-Locked Loop (PLL). The PLL consists of three fundamental stages:

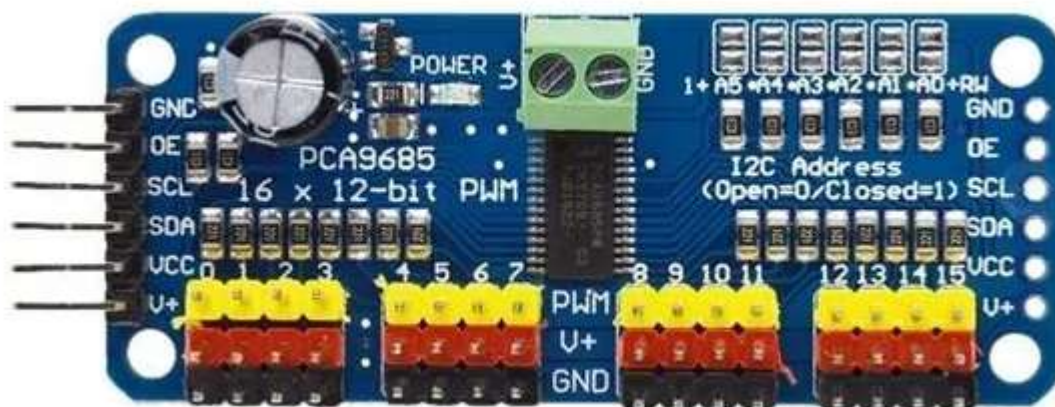
4. **Phase Detector:** Compares the phase of the incoming analog signal with the phase of its local oscillator.
5. **Loop Filter:** Filters the error voltage resulting from the phase detector, removing high-frequency noise and determining the system's dynamic response time.
6. **Internal VCO:** Adjusts its own frequency to match (or "lock" onto) the input signal. Once locked, an internal comparator toggles the logic state of its output (Pin 7) depending on whether the input frequency is higher or lower than the oscillator's free-running frequency.



14 Lead CDIP, PDIP (0.300")

3.4. Kinematic Control via I2C (PCA9685)

Standard servomotors determine their positioning angle based on the width of an electromagnetic pulse (PWM) sent at a strict frequency of 50 Hz (one pulse every 20 ms). Generating 4 precise PWM signals via software would drain the Arduino's timer resources. Therefore, the PCA9685 chip is utilized. It receives commands over the I2C bus (using only two wires: SDA and SCL) and generates hardware signals with a 12-bit resolution (4096 precision steps), offloading the main microcontroller.



4. Equipment and Materials

Control and Processing Hardware:

- 1x Computer with NI LabVIEW software and USB webcam.
- 2x Arduino Microcontrollers (UNO/Nano Boards).
- 1x PCA9685 servo driver module (I2C).
- 4x 180° Servomotors (Base, Shoulder, Elbow, Gripper).

FSK Modulator Components (XR2206):

- 1x Integrated Circuit XR2206.
- 1x Capacitor (Código 103).
- 1x Resistor $R_1 = 50k\Omega$
- 1x Resistor $R_2 = 100k\Omega$
- 1x Resistor variable $R_1 = 50k\Omega$
- 2x Resistors $R_1 = 5.1k\Omega$
- Decoupling capacitors: $C = 1\mu F$ y $10\mu F$

FSK Demodulator Components (XR2211 - Bell 202 Configuration):

- 1x Circuito Integrado XR2211.
- Oscilador Central: $C_0 = 27nF$, $R_0 = 20k\Omega$,
- Potenciometro de ajuste fino $R_x = 20k\Omega$
- Filtro de Lazo: $R_1 = 35.2k\Omega$, $C_1 = 2.7nF$
- Filtro de Datos: $R_F = 178k\Omega$, $C_F = 1nF$
- Histéresis y Salida: $R_B = 1.8M\Omega$, Pull – up $R_L = 5.1k\Omega$
- Capacitor de acoplo de entrada: $0.1\mu F$

5. SAFETY

Ground and Power Isolation:

The PCA9685 module and servomotors must be powered by an external regulated power supply. Drawing power from the Arduino would cause resets due to inductive spikes. Unifying the GND connections of all power sources is mandatory to establish a common reference.

Software Kinematic Limits:

The constrain() function was implemented in the code to delimit minimum and maximum angles (e.g., Gripper between 20° and 90°), preventing mechanical damage to the gears caused by collision.

6. Procedure

6.1. Vision Stage (LabVIEW)

The core of the data processing and decision-making was programmed in the NI LabVIEW graphical development environment, utilizing the NI-IMAQ (for image acquisition) and NI-VISA (for serial communication) modules. To guarantee optimal performance and prevent bottlenecks between heavy video processing and data transmission, the Virtual Instrument (VI) code was structured on a Synchronized Parallel Loops architecture.

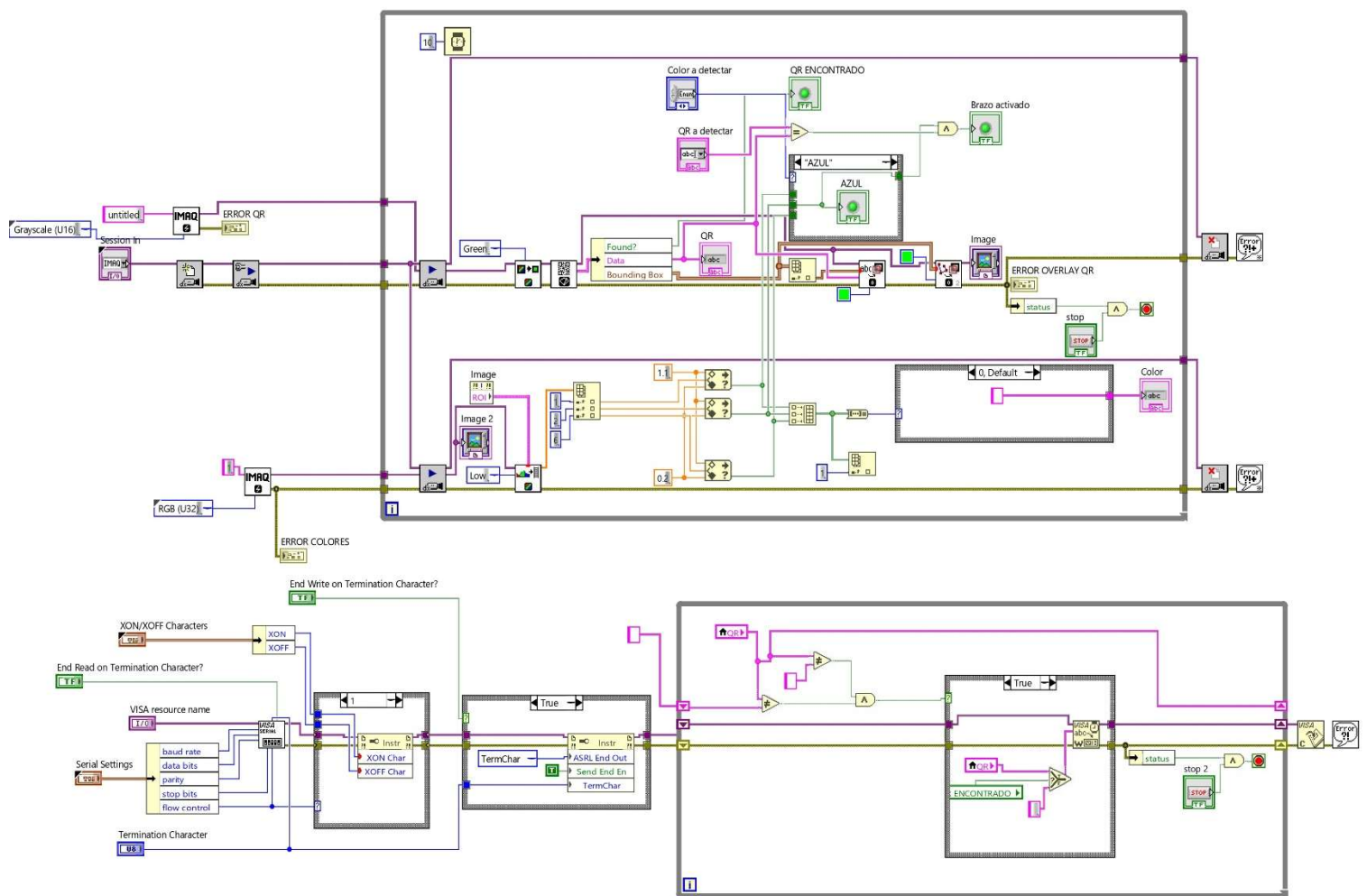
The block diagram (VISION_P1.vi) is fundamentally divided into two While loops executing simultaneously: the Image Processing Loop (top) and the Serial Communication Loop (bottom).

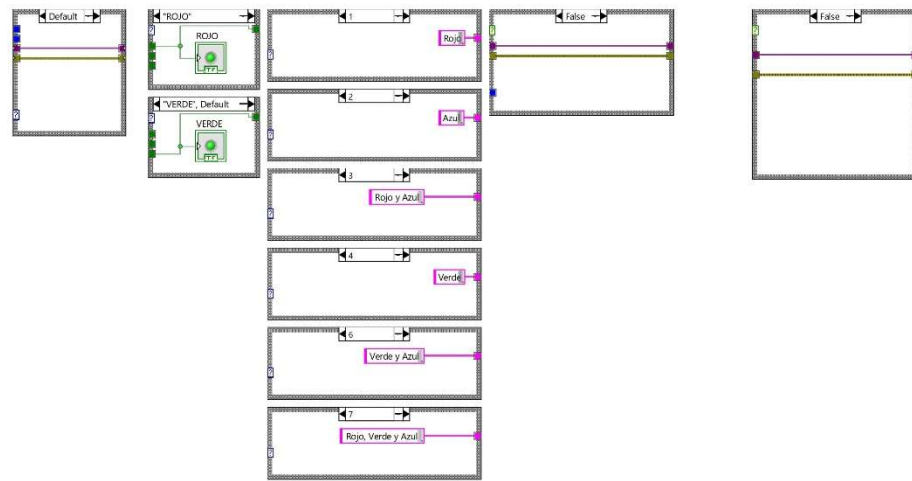
6.1.1. Image Processing Loop (Acquisition and Detection)

This loop is responsible for acquiring camera frames in real-time and executing the machine vision algorithms. It consists of the following stages:

- Initialization (Outside the loop): The *IMAQ Session In* VIs are used to establish communication with the webcam. Two memory spaces are created on the PC using *IMAQ Create*: one configured for grayscale images (Grayscale U16) required for QR reading, and another for color images (RGB U32) used for color discrimination.

- Continuous Acquisition: Inside the main While loop, an *IMAQ Grab* block captures the current frame from the camera buffer and feeds it to two parallel inspection routines.
- **QR Code Detection Routine:**
 - The image enters the *IMAQ Read QR Code* block.
 - This algorithm scans the pixel matrix looking for standard QR alignment patterns.
 - If the read is successful (*Found? = True*), it extracts two critical data points: the decoded text string (*Data*) and the spatial coordinates of the code within the image (*Bounding Box*).
- **Color Sorting Routine:**
 - In parallel, the image goes through a chromatic segmentation algorithm. A specific color plane is extracted (for example, the Green plane extracted with *IMAQ ExtractColorPlanes*) and subjected to particle analysis.
 - The numerical result of this analysis is connected to a Case Structure. This structure acts as a logical multiplexer: depending on the detected color index, it selects and outputs a String constant. The main cases output the labels "ROJO", "VERDE", and "AZUL".
- **Data Fusion and Overlays (User Interface):**
 - The outputs from the previous routines are sent to *IMAQ Overlay* blocks. These superimpose non-destructive visual information over the live video feed (Front Panel), drawing rectangles around the detected QR and displaying floating text with the classified color, easing system debugging.
 - The resulting data (e.g., the detected QR string) is written into local variables (such as ENCONTRADO and Color) so that the Serial Communication Loop can access them asynchronously.





6.1.2. Lazo de Comunicación Serial (Arquitectura de Control de Flujo)

Raw FSK analog transmission is susceptible to saturation if commands are sent continuously. Since the camera captures dozens of frames per second, if LabVIEW sent the command through the serial port on every loop iteration where the QR is visible, it would overflow the transmitting Arduino's buffer.

To solve this, the bottom loop implements an Edge Detection state machine using Shift Registers.

Port Configuration (Outside the loop):

VISA Configure Serial Port is used. It links to the *VISA resource name* control (e.g., COM1) and sets strict physical layer parameters:

1. Baud rate: 9600
2. Data bits: 8
3. Parity: None
4. Stop bits: 1
5. Flow control: None
6. Enable Termination Character: True (Configured with the hexadecimal code 0A, which corresponds to the newline \n).

- **Variable Reading and Comparative Logic:**

Inside the loop, the local variable ENCONTRADO (coming from the top vision loop) is continuously read.

- **State Change Detection (Shift Register):**

The current value of the detected QR is compared using a Not Equal? boolean node against the value from the immediately previous iteration, which is stored in a Shift Register on the edges of the While loop.

Simultaneously, a second node checks that the current string is not empty (ignoring moments where there are no parts on the conveyor belt).

Both conditions feed into an AND gate. This ensures the logical output is True uniquely and exclusively in the split-second a new or different QR appears in front of the camera.

- **Conditioned Transmission (VISA Write):**

The AND gate output controls a Case Structure selector.

If False (the QR is the same or there is nothing), the structure is empty and LabVIEW does not use the COM port, keeping the serial bus free.

If True (a new object to be sorted was detected), the structure concatenates the detected string (e.g., QR_VERDE_OBJ) with the standardized termination character and feeds it to the VISA Write block. This sends a single, clean data packet to the Transmitter microcontroller.



Ilustración 1. Frontal PANEL LABVIEW

6.2.Modulation Stage (TX Transmitter)

El microcontrolador transmisor (TX) actúa como puente entre la computadora y el XR2206.

The transmitting microcontroller (TX) acts as a bridge between the computer and the XR2206.

- **Reception:** The Arduino reads the native serial port at 9600 baud. It stores the characters sent by LabVIEW in a Buffer until it detects the newline (\n), triggering the logical flag dataReady.

- **Synchronization:** Before sending the payload, it injects a prolonged silence and two synchronization characters (~). This gives the receiver's PLL (XR2211) time to properly lock onto the audio signal before critical data arrives.
- **Physical Transmission:** The algorithm breaks down each ASCII letter into 8 bits. It sends pulses to Pin 8 at an exact rate of 62.5 milliseconds per bit. By using inverted logic, the XR2206 receives TTL voltages, instantly shifting its frequency. Thus, the bit stream is transformed into sinusoidal variations of 1 KHz and 2 KHz.

$$f_1 = \frac{1}{R_1 C} = \frac{1}{50k\Omega \times 10nF} = 2000 \text{ Hz}$$

$$f_2 = \frac{1}{R_2 C} = \frac{1}{100k\Omega \times 10nF} = 1000 \text{ Hz}$$

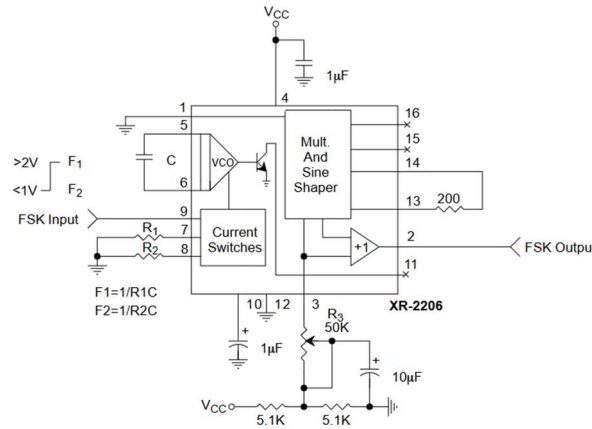


Ilustración 2 Circuit of XR2206

6.2.1. Transmitter Code Analysis (Physical and Data Link Layer)

The transmitting Arduino acts as an interface between the high-speed USB bus (9600 baud) from LabVIEW and the ultra-low-speed analog bus (~16 baud) required by the XR2206 modulator.

To achieve this deterministically, the code is structured into specific routines for acquisition, packaging, and software modulation (Bit-Banging).

```
// PIN AND TIMING CONFIGURATION (PHYSICAL LAYER)
```

```
#define FSK_OUTPUT_PIN 8
```

```
#define BIT_PERIOD_MS 62UL
```

```
#define BIT_PERIOD_US 500UL
```

```
#define INTER_CHAR_DELAY_MS 500UL
```

```
#define INTER_WORD_DELAY_MS 2000UL
```

```
#define MAX_BUFFER_LENGTH 64
```

```
#define SYNC_CHARACTER '~'
```

```
String txBuffer    = "";
```

```
bool  dataReady    = false;
```

```
bool  isTransmitting = false;
```

```
void sendBit(bool bitValue) {
```

```
    digitalWrite(FSK_OUTPUT_PIN, bitValue ? LOW : HIGH);
```

```
    delay(BIT_PERIOD_MS);
```

```
    delayMicroseconds(BIT_PERIOD_US);
```

```
}
```

```
void transmitCharacter(char character) {
```

```
    uint8_t dataByte = (uint8_t)character;
```

```
    sendBit(0); // START BIT
```

```
    for (int i = 7; i >= 0; i--) {
```

```
        sendBit((dataByte >> i) & 1); // 8 DATA BITS
```

```
    }
```

```
    sendBit(1); // STOP BIT
```

```
}
```

```
void readSerialPort() {
```

```
    while (Serial.available()) {
```

```
        char readChar = (char)Serial.read();
```

```

    if (readChar == '\n' || readChar == '\r') {
        if (txBuffer.length() > 0) dataReady = true;
    } else {
        if (txBuffer.length() < MAX_BUFFER_LENGTH) {
            txBuffer += readChar;
        }
    }
}
}

```

```

void setup() {
    pinMode(FSK_OUTPUT_PIN, OUTPUT);
    digitalWrite(FSK_OUTPUT_PIN, LOW);
    Serial.begin(9600);
}

```

```

void loop() {
    readSerialPort();

    if (dataReady && !isTransmitting) {
        isTransmitting = true;
        dataReady = false;

        digitalWrite(FSK_OUTPUT_PIN, LOW);
        delay(INTER_WORD_DELAY_MS);

        transmitCharacter(SYNC_CHARACTER);
        digitalWrite(FSK_OUTPUT_PIN, LOW);
        delay(INTER_CHAR_DELAY_MS);
        transmitCharacter(SYNC_CHARACTER);
        digitalWrite(FSK_OUTPUT_PIN, LOW);
    }
}

```

```

    delay(INTER_CHAR_DELAY_MS);

    for (int i = 0; i < (int)txBuffer.length(); i++) {
        char currentLetter = txBuffer.charAt(i);
        transmitCharacter(currentLetter);
        digitalWrite(FSK_OUTPUT_PIN, LOW);
        if (i < (int)txBuffer.length() - 1) delay(INTER_CHAR_DELAY_MS);
    }

    digitalWrite(FSK_OUTPUT_PIN, LOW);
    delay(INTER_WORD_DELAY_MS);
    txBuffer = "";
    isTransmitting = false;
}
}

```

Algorithm Functional Breakdown:

1. **Physical Constants Definition (Macros):** The bit transmission rate requires holding the electrical line for exactly 62.5ms. This extremely slow speed is an unavoidable hardware requirement; the FSK modem's oscillator circuits and filters need time to stabilize the waveform frequency before the next logic transition occurs.
2. **Asynchronous Acquisition (readSerialPort):** Acts as a circular receive buffer. It appends characters to the txBuffer string until a newline (\n) is detected, triggering the dataReady flag.
3. **Software Modulation (sendBit):** Uses inverted logic (bitValue ? LOW : HIGH) to interface with the XR2206's FSK input (Pin 9), ensuring a logical "1" correctly maps to the hardware "Mark" frequency.
4. **UART Packaging (transmitCharacter):** Packages raw data into a UART frame by sending a Start Bit (0), shifting 8 data bits (MSB first) using bitwise masking (dataByte >> i) & 1, and concluding with a Stop Bit (1).

- 5. Synchronization and Main Sequencer (loop):** Injects a 2000 ms guard time between words to prevent analog packet collisions. Before sending the payload, it transmits the ~ character twice as a Training Frame, allowing the XR2211's PLL at the receiving end to lock onto the carrier signal.

6.3. Demodulation and Control Stage (RX Receiver)

The analog signal travels through the wire and enters the XR2211, which restores the original frame to square pulses on Pin 7. The receiving microcontroller connected to Pin 3 executes a rigorous software algorithm:

- **Debounce Filter:** Samples the signal repeatedly to filter out static spikes.
- **Synchronization:** Blocks the processor until a falling edge is detected.
- **Center of Bit Extraction:** Pauses 1.5 times the bit period to land exactly on the geometric center of the first data bit's "Eye Diagram", reading the signal at peak analog stability.
- **Kinematics:** Once the sentence is assembled, `indexOf()` evaluates if it belongs to a valid instruction, triggering the routine to guide the robot's base to 40°, 110°, or 140° coordinates.

6.3.1. Receiver Code Analysis (FSK Decoding and Kinematic Control)

The receiving microcontroller has a dual critical responsibility: software decoding the digitized audio signal from the XR2211 demodulator and translating that information into safe mechanical trajectories for the robotic manipulator.

```

#include <Wire.h>

#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pwmController = Adafruit_PWMServoDriver();

#define BASE_CHANNEL 0
#define SHOULDER_CHANNEL 1
#define ELBOW_CHANNEL 2
#define GRIPPER_CHANNEL 3
#define PULSE_MIN 150
#define PULSE_MAX 600

#define BASE_LIMIT_MIN 0
#define BASE_LIMIT_MAX 149
#define BASE_HOME 90
// [Other limits omitted for brevity, refer to full code]

int currentAngles[4] = { BASE_HOME, SHOULDER_HOME, ELBOW_HOME,
GRIPPER_HOME };

long mapAngle(long angle) {
    return map(angle, 0, 180, PULSE_MIN, PULSE_MAX);
}

void moveServoSmoothly(uint8_t channel, int targetAngle, uint8_t speed_ms) {
    int currentAngle = currentAngles[channel];
    int validatedAngle;

```

```

    if (channel == BASE_CHANNEL) validatedAngle = constrain(targetAngle,
BASE_LIMIT_MIN, BASE_LIMIT_MAX);

    // [Other constraints omitted for brevity]

    else return;

    if (currentAngle < validatedAngle) {
        for (int a = currentAngle; a <= validatedAngle; a++) {
            pwmController.setPWM(channel, 0, mapAngle(a));
            delay(speed_ms);
        }
    } else if (currentAngle > validatedAngle) {
        for (int a = currentAngle; a >= validatedAngle; a--) {
            pwmController.setPWM(channel, 0, mapAngle(a));
            delay(speed_ms);
        }
    }
    currentAngles[channel] = validatedAngle;
}

// [Kinematic Routine and UART Demodulator logic omitted for brevity, refer to full
repository files]

void loop() {
    if (waitForStartBit(60000UL)) {
        String receivedWord = receiveWordAuto();
        if (receivedWord.length() > 0) {
            if (receivedWord.indexOf("QR_VERDE_OBJ") >= 0) {
                objectSortingRoutine(40);
            }
        }
    }
}

```

```

else if (receivedWord.indexOf("QR_ROJO_OBJ") >= 0) {
    objectSortingRoutine(140);
}
else if (receivedWord.indexOf("QR_AZUL_OBJ") >= 0) {
    objectSortingRoutine(110);
}
}
}
}
}
}

```

System Architecture Analysis:

1. **Hardware Abstraction and Physical Constraints:** Servomotor control is handled via the I2C protocol to the PCA9685 module. Safe operating windows were defined using macros (e.g., GRIPPER_LIMIT_MIN 20). The currentAngles state vector stores real-time positions for interpolation calculations.
2. **Kinematics and Trajectory Stabilization:** The moveServoSmoothly() function prevents abrupt step changes that generate inductive current spikes. It uses constrain() for safety and linear interpolation (for loops) to emulate a constant velocity profile. The parameterized objectSortingRoutine acts as a finite state machine for the Pick-and-Place sequence.
3. **FSK Demodulation and UART Emulation:** Implements Software UART. Functions include readDigitalAverage (Oversampling technique to filter static), waitForStartBit (Conditional blocking loop for falling edge detection), and receiveCharacter (Timing alignment to sample the "Center of the Eye" for maximum noise margin).
4. **Main Loop:** Uses defensive programming via .indexOf(). If thermal transients in the VCO introduce garbage characters at the message periphery, the algorithm filters out the noise and strictly seeks the control tags.

7. Results

The system operated stably end-to-end:

- **Detection and Link:** Serial transmission injected bits with strict logical pauses (Preamble with ~ characters), stabilizing the demodulator.
- **Signal Validation:** Spurious variations on the analog bus were successfully mitigated by the XR2211's capacitive filter hysteresis and software validation (Stop Bit kept HIGH).
- **Kinematic Response:**
 - Reading QR_VERDE_OBJ: Frontal routine execution, left deposit at 40°.
 - Reading QR_AZUL_OBJ: Frontal routine execution, intermediate deposit at 110°.
 - Reading QR_ROJO_OBJ: Frontal routine execution, right deposit at 140°.

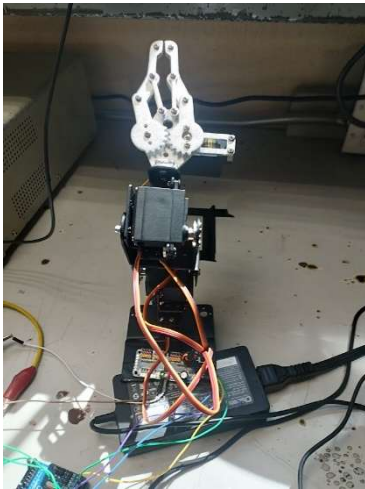


Ilustración 4. Robotic arm. 4 degrees of freedom



Ilustración 3. Final Project

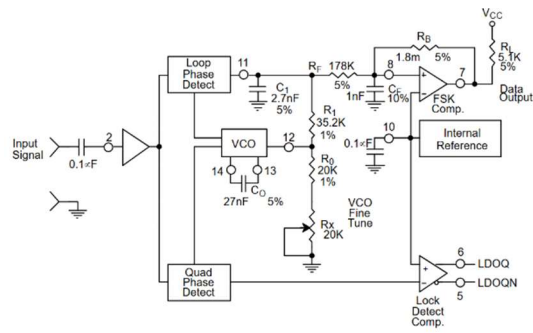


Ilustración 5. esquematic XR2211

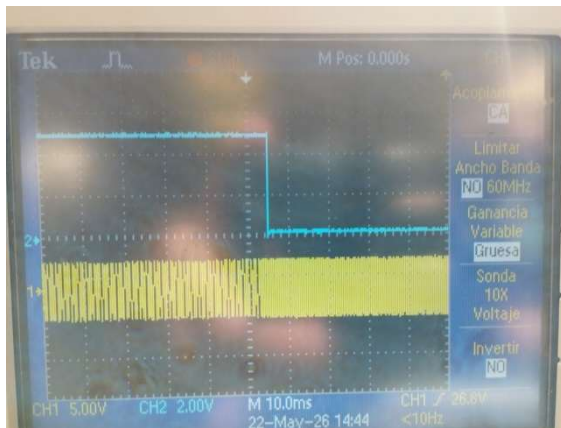


Ilustración 7. Modulation FSK XR2206

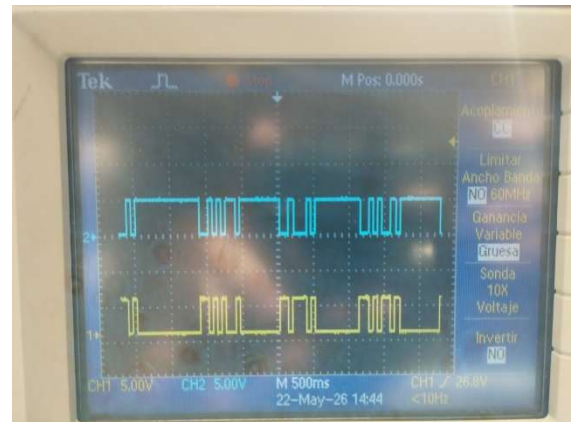


Ilustración 6. demodulation FSK. XR2211

8. Conclusiones y recomendaciones

The integration of computer vision systems and analog transmission demonstrates the critical sensitivity of digital timing. The primary challenge was the rigorous coupling of the software-emulated Baud Rate (62.5 milliseconds per bit); any timing offset introduced by blocking functions or heavy operations caused shifted and erroneous readings.

It is recommended to use shielded cables for the FSK line if spanning long distances is required, preventing parasitic capacitances that deform the sinusoidal tones. In future iterations, UART emulation ("bit-banging") could be replaced by hardware Timer Interrupts, drastically optimizing the microcontroller's processing overhead.

9. References

1. Exar Corporation. (1997). *XR-2206 Monolithic Function Generator*. Datasheet.
2. Exar Corporation. (1997). *XR-2211 FSK Demodulator / Tone Decoder*. Datasheet.
3. NXP Semiconductors. (2015). *PCA9685 16-channel, 12-bit PWM Fm+ I2C-bus LED controller*. Product data sheet.
4. National Instruments. (2022). *NI Vision Concepts Manual*. Conceptos teóricos de adquisición y procesamiento de imagen para sistemas de control.